Empowering IETF Collaboration with NLP Search Innovations and LLM-Enhanced RFC Writing

Jie Bian
University of Oslo
jiebi@ifi.uio.no

Michael Welzl
University of Oslo
michawe@ifi.uio.no

Abstract

The Internet Engineering Task Force (IETF) produces extensive textual data, including discussions in email archives and GitHub repositories and Internet-Drafts (I-Ds), which are preliminary versions of Requests for Comments (RFCs). The sheer volume and complexity of this material present significant workflow challenges, contributing to the duration of the standardization process (it is common to take several years from an initial draft to the final RFC). This paper explores the potential of Natural Language Processing (NLP) using Large Language Models (LLMs) to streamline IETF workflows. We use Information Retrieval (IR) to i) build a search system that helps users locate comments related to details in an I-D, and ii) partially automate RFC writing.

CCS Concepts

• **Networks** → **Network protocols**; • **Information systems** → **Presentation of retrieval results**; • **Computing methodologies** → **Natural language generation**.

Keywords

IETF, RFC, I-Ds, LLM, Information Retrieval, Instruction Tuning, Test Time Scaling

ACM Reference Format:
Jie Bian and Michael Welzl. 2025. Empowering IETF Collaboration with NLP Search Innovations and LLM-Enhanced RFC Writing. In *Applied Networking Research Workshop (ANRW 25), July 22, 2025, Madrid, Spain.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3744200.3744761

1 Introduction

To date, nearly 10,000 RFCs have been published, a remarkable achievement made possible by substantial human effort [39]. This includes writing or reviewing I-Ds, providing feedback, engaging in communication via email or GitHub, and ensuring that all aspects, such as IETF guidelines and relationships to other standards, are thoroughly considered.

These efforts are commonly spread over several years. This can lead to several issues. Firstly, discussions can become fragmented and disorganized as communications are scattered across a massive mail archive or buried in personal mailboxes. The vast and unstructured nature of the IETF email archive, coupled with its constant growth, creates a significant challenge for locating specific discussions or proposals, often requiring time-consuming manual sifting through irrelevant or outdated content. In addition, the journey from the initial draft to the final RFC can be long, often spanning several years. During this period, some drafts may encounter significant delays or even stall midway due to, for example, author affiliation changes or personal breaks such as a parental leave. A prolonged timeline can impede timely decision-making and the implementation of new standards.

This led us to explore whether LLMs could assist with this process, given their impressive ability to excel not only in natural but also in highly specialized languages, such as chemistry molecules [41], mathematical equations [3], and programming codes [21]. I-Ds or RFCs, in many ways, are closer to natural languages, which suggests that LLMs potentially could enhance the RFC definition and publication workflow. We intend to leverage it to streamline the drafting process, alleviate the burden on human contributors, and potentially accelerate the transition from initial drafts to finalized RFCs.

The IETF community openly publishes all documents online. While these resources are abundant, e.g., the mail archive, using these raw data directly is not feasible for our purposes. To obtain the necessary dataset, we leveraged GitHub repositories and compiled a collection of about 200 I-Ds. The idea behind this is that GitHub provides us with the relationships needed to bridge human feedback and the iterations of I-Ds, so that we can build a dataset that reflects the history of I-D editing. We call this dataset *TextFlow*.

There are several ways of exploiting LLM for this purpose. We have attempted to associate comments with specific text snippets within the I-Ds or RFCs, which helps to establish connections between the I-Ds' detailed descriptions and human input. The approach used here is information retrieval (IR). Based on existing available LLMs that excel in IR, we



This work is licensed under a Creative Commons Attribution 4.0 International License.
ANRW 25, Madrid, Spain
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2009-3/25/07
https://doi.org/10.1145/3744200.3744761

have applied them first and then tuned them on our curated dataset *TextFlow* to be domain-adapted. We envision that the usage of our *TextFlow Search* (IR) in the IETF email archive could help streamline the discovery of critical discussions for certain design decisions, identify recurring, unresolved, or controversial issues, and uncover historical context that informs current standards development.

Additionally, we utilize LLMs to automate I-D text based on previous versions and associated comments—we call this *TextFlow Edit*. We developed tailored prompts to guide LLMs in generating revised I-D text during inference, and applied instruction tuning to enhance the model's general capabilities by incorporating task-specific instructions during training. To summarize, our contributions are:

(1) *TextFlow*, a curated dataset which includes the WG discussions alongside the corresponding changes in I-Ds (before and after).
(2) *TextFlow Search*, an IR-based search system that links these comments to specific I-D details.
(3) *TextFlow Edit*, a generative task where we instructed the LLM to produce specific parts of the I-Ds based on the previous versions and the associated comments.

Integrating LLMs into technical documentation workflows reduces the time and effort required for drafting and revising documents, accelerating RFC publication while maintaining and enhancing the quality of standards. We therefore believe that our framework represents a significant step towards AI-driven collaboration in technical documentation, offering a new pathway for future advancements in the field.

2 Related Work

Research on the use of LLMs for networking is experiencing significant growth, especially in areas where LLMs are integrated for network design and planning, implementation, analytics, and management [11]. LLMs can even produce usable protocol data, albeit with limitations [2]; this could be used to diagnose errors.

A few studies have also focused on IETF text material [32]: McQuistin et al. [19] evaluate methods for modeling and parsing protocol data, proposing a typed representation for improved correctness and usability; Yen et al. [39] highlight NLP's potential to reduce human supervision and errors in drafting protocols. *Sage* [40] is an NLP system for semi-automated protocol implementation generation, and protocol primitives are also extracted from RFCs by *PROSPER* [29].

To the best of our knowledge, Bian et al. [5] presents the only earlier work that investigates the relationship between IETF discussions and I-D text. This preliminary study indicates that a system such as *TextFlow Search* could, in principle, serve as a basis for education. This study, with its approach to utilize GitHub data, has inspired the present paper.

3 The TextFlow Dataset

TextFlow consists of two major elements: i) the various versions of I-Ds, and ii) comments that have led to textual changes in them.

As of this writing, the formal RFC count has nearly reached 10 000. For preliminary versions of I-Ds, the number is significantly higher, as RFCs commonly advance through a series of revisions, beginning with version 00 and escalating to version 15 or higher as needed. Based on a rough estimate from 20 RFCs published in 2018 and analyzed in RFC 8963 [13], the average RFC is approximately 35 pages long, and the average delay from initial draft submission to final publication is around 1 182 days. These figures highlight the substantial effort and collaboration involved in bringing a protocol specification to completion. The 200 I-Ds in our curated repositories span various stages of development, ranging from early drafts with active discussion and major revisions to near-final versions focused primarily on editorial and grammatical refinements. This variability adds complexity to our tasks. Of the 200 I-Ds in our collection, 160 were gathered before Dec. 2024 and are utilized for training, evaluation, and testing. The remaining 40, collected after this date, serve as out-of-domain (OOD) data. By "OOD," we mean there are no overlaps with the initial 160 I-Ds. The OOD set includes topics or sub-domains not represented in the initial 160 RFCs. However, all 200 I-Ds belong to the broader "RFC" domain.

Historically, most comments and reviews occurred within the IETF's extensive email archives. However, leveraging this resource effectively has been challenging due to the lack of explicit connections between the comments and the specific sections of an I-D or RFC they refer to. Identifying these relationships is labor-intensive and highly demanding, typically requiring the expertise of the authors or members of the corresponding WG who are familiar with the document's details. Recently, GitHub has emerged as a collaborative platform where authors can raise an issue, receive community feedback, and refine their I-Ds. By leveraging the mechanisms of Pull Requests (PRs) and Issues, we can establish these relationships through GitHub's embedded features. It allows us to track changes and comments, linking comments directly to specific sections of the I-Ds. Some issues include a URL to specific emails, such as "We've discussed it before, please see URL." This explicit mention has allowed us to utilize a small portion of the emails. We've extracted text snippets from I-Ds within the comments, if they exist.

The data we obtain from GitHub is initially in `diff` format, similar to that used in code repositories. This format includes the changed lines of the I-Ds text snippets along with the surrounding context lines. Additionally, we separate this `diff` into text pairs representing the pre- and post-versions of the I-Ds. For the *TextFlow Search* task, we currently use the

`diff` format of I-Ds. However, depending on the purposes, we could alternatively use either the pre- or the post-version. As mentioned above, we divided the first 160 repositories we curated based on Pull Requests, with 80% for training, 10% for development, and 10% for testing, see Table 1.

Table 1: Quantitative Summary of TextFlow

Number	Train	Dev	Test	OOD
Entries	4 196	1 221	1 299	4 307
Comments	4 185	539	521	1 534
Text Snippet (I-Ds)	10 152	1 221	1 297	4 063

4 TextFlow Search

Information retrieval (IR) quickly and efficiently obtains relevant information from vast collections of data. Traditionally, IR uses the BM25 [28] algorithm, which ranks documents based on term frequency and inverse document frequency. Emerging dense retrieval [15] techniques utilize LLMs to encode the semantic context of queries and documents. These methods rely on neural networks to convert text into dense vector embeddings, allowing relevance judgments to be made with greater precision through semantic similarity analysis.

4.1 Sparse and Dense Retrieval

We adhere to established norms in IR and have developed a search framework capable of tracking changes in I-Ds text snippets to corresponding comments, and vice versa. For dense retrieval, we respectively explored BGE [6, 37], GritLM [22], Promptriver [36], and E5 [34, 35] that demonstrate remarkable performance in the IR benchmark MTEB [7, 23]. These models can be classified into two categories: transformer encoder and decoder-based models [33]. Encoders excel at encoding semantics into static representations using bidirectional context, while decoders unfold semantics sequentially, learning meaning through prediction. We use mean pooling of embeddings from encoder models and the last token's representation from decoder models to capture the semantic meaning of text. These textual embeddings are then used to compute the semantic similarity between a query and a document, which could be pairs of text snippets from an I-D (query) and comments (document) or vice versa.

4.2 Supervised Fine-Tuning (SFT)

Building on the success of the aforementioned models, we employ BGE-large [37] and Mistral-7B [14] as our base models, fine-tuning them on our dataset using contrastive loss [6], which improves search accuracy by training models to distinguish between relevant and irrelevant items. During training, the model processes item pairs—positive (query and relevant document) and negative (query and irrelevant document)—learning to minimize distances for positive pairs and maximize them for negative pairs. Contrastive loss requires a large batch size to provide sufficient negative samples. To accommodate an LLM like Mistral-7B, we use Parameter-Efficient Fine-Tuning (PEFT) [18] with Low-Rank Adaptation (LoRA) [12]. Additionally, we optimize memory usage by applying techniques such as gradient checkpointing, mixed-precision training [20], and DeepSpeed ZeRO-1 [27].

5 TextFlow Edit

Instruction tuning is essential for enabling LLMs to generate coherent and contextually relevant text, significantly enhancing natural language generation (NLG) quality. In our preliminary investigation of this task, we systematically experimented with various scales of Mistral models (7B [14] and 24B), Llama3 models (8B and 70B [8]), and reasoning models such as Qwen [38] (7B and 32B), QwQ-32B [31], and DeepSeek-R1 [10] distilled models based on Llama-8B, Qwen-32B, and Llama-70B. Building on prior work that identified LLaMA-8B as a representative intuitive general-purpose model and Qwen-32B as a representative reasoning model—along with their derivatives as high-potential candidates—this paper extends those findings through thorough experiments and provides a detailed analysis of the results.

5.1 Models

Llama-8B provides strong contextual understanding, coherent text generation, and the ability to handle diverse writing styles. However, it may struggle with nuanced feedback, complex revisions, and ambiguous feedback, potentially leading to inaccuracies or biases. Therefore, we also employ reasoning models. Qwen-32B demonstrates strong capabilities in multi-step reasoning, effectively breaking down complex tasks and producing well-structured solutions. It emulates human analytical thinking by recognizing patterns, evaluating evidence, and integrating insights across diverse domains. In this paper, all models except Llama-8B are of this type. We refer to DeepSeek-R1 [17] distilled models as R1-(Model).

5.2 Inference

In our experiments, we designed a prompt to evaluate the impact of human feedback on text revision. The prompt guides LLMs in generating revised text by integrating the original text and the provided feedback. We opted for 4-bit quantization for all the aforementioned models for faster inference and less memory usage, given the limitations of available hardware resources. Specifically, we set the maximum input length to 2048 tokens and allow for a maximum of 512 generated new tokens. To maximize the generative capability of LLMs, we use recommended hyperparameters like temperature and penalty for each model in this paper.

5.3 Testing Time Scaling (TTS)

Allocating extra time and computational resources to LLMs during inference can improve results more effectively than adjusting their parameters [30]. TTS enables deeper reasoning and thorough analysis, enhancing the model's ability to handle complex tasks without extensive retraining or architectural changes. Inspired by this, we adopt a straightforward strategy outlined in [24], encouraging the model to explicitly articulate its thought process using the word "think,", which we refer to as TTS I, and also an extended thought process which is followed by an additional self-checking word "wait,", which we refer to as TTS II, and finally let it conclude the final answer. We allow a maximum of 4096 tokens for the thought process.

5.4 Instruction Tuning (SFT)

Instruction tuning [42] is a distinct type of supervised fine-tuning (SFT) featured by training the model with pairs of instructions and their corresponding outputs. In this context, instructions are the specific guidelines or commands provided to the model, detailing what we want the model to do, e.g., to summarize the key points from a news article or generate a Python function to sort a list. Besides the instruction itself, additional contextual information can also be provided, e.g., in coding tasks, instruction tuning datasets [1] include programming questions, solutions, test cases, execution feedback, and quality assessments. In our task, we ask the LLM to take an old version and the feedback to generate a revised version.

6 Results and Analysis

6.1 TextFlow Search

IR Metrics. Mean Reciprocal Rank (MRR), Normalized Discounted Cumulative Gain (NDCG), and Recall are essential metrics for evaluating IR systems [26]. MRR measures the efficiency of retrieving the first relevant document by focusing on the reciprocal rank of the highest-ranked relevant item across queries. It prioritizes systems that deliver at least one relevant result quickly, reflecting how effectively a system surfaces the best result early in the ranking. NDCG evaluates the quality of ranked results by combining relevance scores (e.g., 2, 1, 0) and their positions, applying a discount factor to penalize lower-ranked items. It emphasizes accurate ordering, ensuring the most relevant document is ranked highest, and accounts for the value of multiple relevant documents. Recall quantifies completeness by measuring the proportion of relevant documents retrieved relative to all relevant documents in the dataset. In our task, MRR is prioritized because our focus is on delivering at least one high-quality result swiftly, and we treat relevance as a binary criterion (relevant vs. not relevant). Nevertheless, we present all results to provide a comprehensive view of our search system. All evaluation metrics are scaled from 0 to 1. However, the reported values are multiplied by 100 for easier interpretation.

IR Results. We present the results in Table 2. We retrieve the top 1k documents per query and evaluate performance using the above metrics at rank 3 (e.g., MRR@3). An intuitive explanation is that among the top 3 ranked results, our targeted item (comment) is often found in the second position (1/2) according to the MRR metric, while using the I-Ds text snippet as query (q:i-ds). For instance, our best model, a fine-tuned version of the Mistral-7B base model, achieves an MRR of 64.58%. This indicates that the item is typically between the first (1) and second (1/2) positions on the test set, and between the second (1/2) and third (1/3) positions on the OOD set. This demonstrates a robust performance. When we use comments as queries (q:c), the values of evaluation metrics are even higher, e.g., 78.18% compared to 64.58% (q:i-ds). This aligns with how humans connect details in an I-D with comments. For example, it's easier for us to read comments first and then consider or search for specific design decisions written in the I-Ds that the comments refer to.

Table 2: TextFlow Search Results; best model in bold

Model	Set	Metrics (q:i-ds)			Metrics (q:c)		
		MRR	NDCG	REC	MRR	NDCG	REC
BM25	Test	40.56	38.25	39.13	61.61	55.21	52.78
	OOD	22.51	21.26	23.26	39.44	32.97	31.50
BGE	Test	40.72	38.42	39.35	59.47	53.98	52.22
	OOD	22.88	21.65	23.86	37.15	31.16	30.32
BGE (SFT)	Test	51.98	50.32	52.02	71.11	64.84	61.23
	OOD	29.96	29.06	32.33	46.14	39.39	37.81
Promptriever (Mistral-based)	Test	39.09	36.98	37.41	67.72	61.12	58.02
	OOD	22.72	21.31	23.09	45.22	38.10	36.22
Mistral (SFT)	Test	64.58	63.27	64.67	78.18	73.20	67.19
	OOD	41.12	40.10	43.55	55.94	48.33	45.11

6.2 TextFlow Edit

NLG Metrics. We assess LLM capabilities using established metrics from multiple perspectives. BLEU [25] measures N-gram overlap for surface-level accuracy. BertScore [43] evaluates semantic meaning through contextual embeddings. METEOR [4] considers exact matches and semantic similarity for nuanced evaluation. Mauve [9] analyzes the statistical divergence between generated text and human-written documents, emphasizing human-like writing characteristics. It evaluates distributional alignment to determine whether the text adheres to the same genre, topic, or stylistic features as the reference. All reported values are also multiplied by 100.

NLG Results. We demonstrate the results on the test set in Table 3 and Table 4, and the OOD set in Table 5. To aid interpretation, different colors are used to indicate comparison types: black represents comparisons between generated text

and the new human-written version, gray denotes comparisons with the older version, and brown highlights comments. Abbreviations include o(old), c(comment), n(new), GT. for generated text, and RT. for referenced text. A dash ("-") indicates values less than 1e-2, which can be ignored.

We count the number of successfully generated outputs when processing 1,299 test samples. Some reasoning models, like Qwen-32B, may not consistently produce final outputs for all input samples. While these models can process revised text internally during their reasoning (while using TTS, the model's internal reasoning steps are exposed, clarifying this), they occasionally fail to reach a definitive conclusion, resulting in the revised text not being extracted from the thought process. This does not indicate an inherent inability to generate results; when applied effectively (e.g., with targeted techniques such as TTS), they successfully derive conclusions and produce revised text. It also shows that TTS I seems enough, and TTS II with self-checking is not necessary, as it hurts the performance metrics. Performance metrics were calculated based solely on successfully generated outputs.

The revised text generated by the LLM tends to resemble either the original or the updated I-D snippet more closely than the associated comments. Although some comments explicitly propose edits, such instances are relatively infrequent. When comments include quoted I-D text, we extract it to prevent the model from leveraging it directly during the search task. Consequently, comments exert limited influence on similarity metrics—reflected in generally lower scores—except for BERTScore, which remains consistently above 0.8. Overall, the findings indicate that the models follow instructions effectively. In most cases, the revised text aligns more closely with the original version than with the human-written update, though it occasionally achieves higher Mauve scores.

Employing our SFT Llama-8b as a case study, the model-adjusted content more closely resembles the old version in terms of surface-level structure (BLEU), semantic meaning (BERT), and structural/paraphrastic quality (METEOR). This suggests a higher resemblance to the original (old) text than the human-revised new version. However, the model output exhibits greater distributional similarity (MAUVE) with the new version, indicating that it captures the general style and variety of the human-edited enhancements at a macro-level. Despite this, it still struggles to replicate the exact wording or semantic subtleties of the updated material in specific cases. Overall, while the model reflects the general distributional characteristics of the revised version, it falls short in reproducing its finer-grained human-like edits.

Numerical results show that the intuitive model, like Llama-8B, outperforms the larger reasoning model Qwen-32B in both output quantity (see Table 3 and 4) and similarity to the target text (for both test set and OOD). This discrepancy of metric values arises because Qwen-32B, as well as other R1-(Model)s, designed for reasoning, inevitably generate additional content like summaries and explanations in the final output, despite our request for the conclusion to contain solely the revised text. This supplementary material dilutes the revised text's alignment with the desired output, reducing its similarity score. Although Qwen-32B may not reach the same level of alignment with the updated text as SFT Llama-8b, it is still dependable because of its clear explanations and rigorous compliance with user directives, in contrast to human-written text, which can sometimes diverge from feedback as WGs do not incorporate every detail of the requested changes from the comments. Figure 1 presents an actual input and output example.

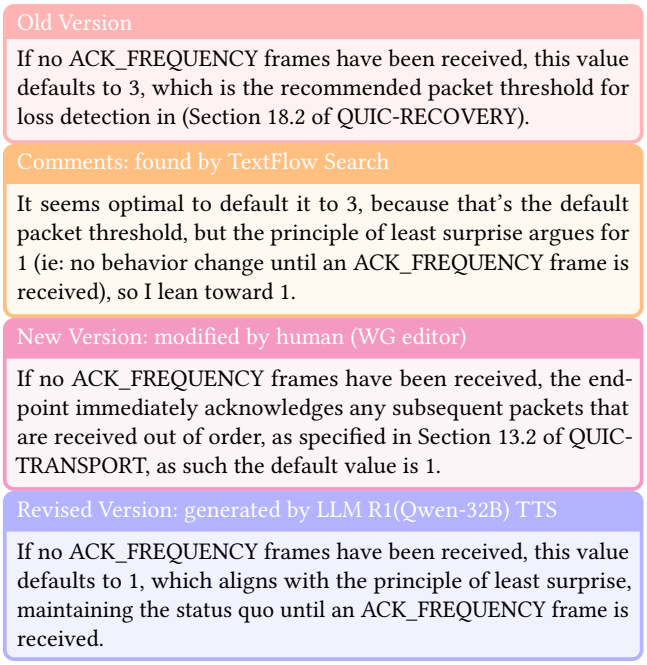


Figure 1: TextFlow Search and Edit in action: with the diff (1st vs. 3rd box) as input, TextFlow Search finds the related comments. Using the old version and comment (1st and 2nd box) as input, TextFlow Edit generated a usable text update (the last box).

Our results reveal that SFT (instruction-tuning without thought process) negatively impacts reasoning models' performance, as enabling SFT results in a measurable decline compared to the model's baseline performance without SFT, e.g., Qwen-32B and Qwen-32B (SFT). This suggests that SFT may help train models not to generate "extra" content, but interfere with the model's reasoning capabilities, possibly by overwriting or conflicting with its pre-trained reasoning mechanisms. However, we lack a clear thought process to fine-tune the model's reasoning capabilities effectively.

Table 3: Comparison of Qwen models across various metrics on the test set (n = 1299)

in → out	NO.	GT./RT.	Metrics			
			BLEU	BERT	METEOR	MAUVE
Qwen-32B						
$o + c \rightarrow n'$	536	n'/n	27.26	91.19	49.78	24.78
		n'/o	39.94	92.37	57.22	25.09
		n'/c	0.03	82.02	9.41	2.74
Qwen-32B (TTS I)						
$o + c \rightarrow n'$	1299	n'/n	30.17	89.25	51.36	10.80
		n'/o	33.97	90.31	57.35	9.92
		n'/c	1.35	83.51	18.77	1.57
Qwen-32B (TTS II)						
$o + c \rightarrow n'$	1299	n'/n	27.59	88.72	49.39	5.73
		n'/o	30.90	89.78	55.06	6.18
		n'/c	1.47	83.38	18.62	1.36
Qwen-32B (SFT)						
$o + c \rightarrow n'$	484	n'/n	8.69	83.90	20.86	13.42
		n'/o	15.52	84.76	24.57	13.21
		n'/c	-	80.33	5.42	5.13
R1 (Qwen-32B)						
$o + c \rightarrow n'$	204	n'/n	33.66	91.48	53.04	70.20
		n'/o	42.94	92.61	58.83	75.60
		n'/c	-	82.10	10.55	3.83
R1 (Qwen-32B) (TTS I)						
$o + c \rightarrow n'$	1197	n'/n	29.01	88.14	43.04	6.47
		n'/o	33.06	88.90	47.76	6.32
		n'/c	0.53	83.39	16.79	1.45
R1 (Qwen-32B) (TTS II)						
$o + c \rightarrow n'$	1255	n'/n	27.80	88.33	46.14	5.24
		n'/o	30.20	89.09	50.73	5.23
		n'/c	0.95	83.40	17.74	1.40
R1 (Qwen-32B) (SFT)						
$o + c \rightarrow n'$	612	n'/n	25.31	86.67	38.59	35.85
		n'/o	35.36	87.81	44.34	35.09
		n'/c	-	79.55	6.55	2.41

Table 4: Comparison of Llama models across various metrics on the test set (n = 1299)

in → out	NO.	GT./RT.	Metrics			
			BLEU	BERT	METEOR	MAUVE
Llama-8B						
$o + c \rightarrow n'$	1278	n'/n	36.86	91.72	54.41	55.93
		n'/o	55.46	93.73	66.46	55.59
		n'/c	0.16	82.22	11.75	2.26
Llama-8B (SFT)						
$o + c \rightarrow n'$	1297	n'/n	47.63	93.33	63.98	94.69
		n'/o	69.58	95.64	77.93	92.25
		n'/c	0.25	81.56	10.86	1.92
R1 (Llama-8B)						
$o + c \rightarrow n'$	397	n'/n	7.30	85.10	23.12	7.65
		n'/o	13.71	86.29	30.00	9.65
		n'/c	-	81.93	6.46	3.43
R1 (Llama-8B) (TTS I)						
$o + c \rightarrow n'$	1202	n'/n	36.02	89.90	50.61	19.24
		n'/o	41.16	91.31	58.23	18.01
		n'/c	1.97	83.18	18.19	2.38
R1 (Llama-8B) (TTS II)						
$o + c \rightarrow n'$	1229	n'/n	30.02	88.73	46.72	10.39
		n'/o	34.04	89.87	52.55	9.97
		n'/c	1.93	83.42	19.74	2.06
R1 (Llama-8B) (SFT)						
$o + c \rightarrow n'$	486	n'/n	17.45	87.39	35.35	27.06
		n'/o	30.11	89.18	45.01	27.69
		n'/c	-	81.93	9.26	9.10

Table 5: Model comparison on OOD set (n = 4307) against the new (human-written) versions; best-performing values in bold, and the second-best underlined

Model	NO.	Metrics			
		BLEU	BERT	METEOR	MAUVE
Llama-8B	4274	35.26	91.97	55.71	57.45
Llama-8B (SFT)	**4307**	**47.01**	**93.66**	**65.50**	**91.29**
R1 (Llama-8B)	1115	7.23	85.26	24.62	9.16
R1 (Llama-8B) (TTS I)	4077	35.77	89.69	49.18	13.85
R1 (Llama-8B) (TTS II)	4023	29.64	88.69	47.30	8.71
R1 (Llama-8B) (SFT)	1375	16.48	87.35	34.35	20.69
Qwen-32B	1761	24.12	90.90	48.68	27.08
Qwen-32B (TTS I)	**4307**	28.37	88.94	50.13	6.76
Qwen-32B (TTS II)	**4307**	26.66	88.54	48.88	3.57
Qwen-32B (SFT)	1597	5.29	83.98	20.04	11.38
R1 (Qwen-32B)	603	<u>42.08</u>	<u>92.75</u>	<u>59.98</u>	<u>67.30</u>
R1 (Qwen-32B) (TTS I)	3922	29.97	88.21	44.36	5.00
R1 (Qwen-32B) (TTS II)	4166	26.52	87.90	45.59	4.43
R1 (Qwen-32B) (SFT)	1892	24.06	86.28	38.38	32.19

7 Conclusion and Future Work

We have introduced *TextFlow Search*, a retrieval system designed to automate the extraction of discussion insights from unstructured communication channels (e.g., GitHub Issues, with plans to include email archives in the future). We have also introduced *TextFlow Edit*, which enables LLMs to streamline iterative documentation refinement by incorporating human-provided feedback (e.g., comments, edits) into technical content. Underlying these two systems, the *TextFlow* database is a cornerstone of our work that can be useful for others who analyze textual IETF material. The dataset used in this study is available at https://huggingface.co/datasets/jiebi/TextFlow.

Our next step will be to integrate the two approaches into one cohesive system, similar to a Retrieval-Augmented Generation (RAG) [16] system. This unified framework will combine the strengths of *TextFlow Search* and *TextFlow Edit*, enhancing the efficiency and accuracy of incorporating feedback into technical documentation. Considering the similarity in textual style, it appears quite feasible that *TextFlow Search* and *TextFlow Edit* could be applied to the IETF email archive. The integrated system could be deployed as a standalone tool during the review and revision stages, such as during WG Last Call. It would proactively search for related comments, examine both the content and draft text, and offer structured editing suggestions to clarify ambiguities or address discrepancies. These suggestions could be surfaced in real time as authors edit, or retrospectively through an ex-post checking mechanism that flags unresolved issues.

Acknowledgments

We acknowledge the use of Microsoft Copilot for its assistance with rephrasing, grammar-checking, enhancing the readability of this manuscript, and naming our datasets.

References

[1] Wasi Uddin Ahmad, Aleksander Ficek, Mehrzad Samadi, Jocelyn Huang, Vahid Noroozi, Somshubra Majumdar, and Boris Ginsburg. 2025. OpenCodeInstruct: A Large-scale Instruction Tuning Dataset for Code LLMs. arXiv:2504.04030 [cs.SE] https://arxiv.org/abs/2504.04030

[2] Jari Arkko, Dag Lindbo, and Martin Klitte. 2024. Do Large Language Models Dream of Sockets?. In *Proceedings of the 2024 Applied Networking Research Workshop*. 103–105.

[3] Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. 2023. Llemma: An open language model for mathematics. *arXiv preprint arXiv:2310.10631* (2023).

[4] Satanjeev Banerjee and Alon Lavie. 2005. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. Association for Computational Linguistics, Ann Arbor, Michigan, 65–72. https://www.aclweb.org/anthology/W05-0909

[5] Jie Bian, Michael Welzl, Andrey Kutuzov, and Nikolay Arefyev. 2024. Tell Me Why: Language Models Help Explain the Rationale Behind Internet Protocol Design. In *2024 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN)*. 447–453. doi:10.1109/ICMLCN59089.2024.10624781

[6] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2023. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. arXiv:2309.07597 [cs.CL]

[7] Kenneth Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, Jay Gala, Wissam Siblini, Dominik Krzemiński, Genta Indra Winata, et al. 2025. Mmteb: Massive multilingual text embedding benchmark. *arXiv preprint arXiv:2502.13595* (2025).

[8] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[9] Jian Guan, Zhexin Zhang, Zhuoer Feng, Zitao Liu, Wenbiao Ding, Xiaoxi Mao, Changjie Fan, and Minlie Huang. 2021. OpenMEVA: A benchmark for evaluating open-ended story generation metrics. *arXiv preprint arXiv:2105.08920* (2021).

[10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[11] Ching-Nam Hang, Pei-Duo Yu, Roberto Morabito, and Chee-Wei Tan. 2024. Large Language Models Meet Next-Generation Networking Technologies: A Review. *Future Internet* 16, 10 (2024). doi:10.3390/fi16100365

[12] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[13] C Huitema. 2021. RFC 8963 Evaluation of a Sample of RFCs Produced in 2018. (2021).

[14] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

[15] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering.. In *EMNLP (1)*. 6769–6781.

[16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[17] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[18] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. 2022. PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. https://github.com/huggingface/peft.

[19] Stephen McQuistin, Vivian Band, Dejice Jacob, and Colin Perkins. 2020. Parsing protocol standards to parse standard protocols. In *Proceedings of the 2020 Applied Networking Research Workshop*. 25–31.

[20] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, et al. 2017. Mixed precision training. *arXiv preprint arXiv:1710.03740* (2017).

[21] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro Von Werra, and Shayne Longpre. 2023. Octopack: Instruction tuning code large language models. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*.

[22] Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. 2024. Generative Representational Instruction Tuning. arXiv:2402.09906 [cs.CL]

[23] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2022. MTEB: Massive Text Embedding Benchmark. *arXiv preprint arXiv:2210.07316* (2022). doi:10.48550/ARXIV.2210.07316

[24] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. arXiv:2501.19393 [cs.CL] https://arxiv.org/abs/2501.19393

[25] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[26] Filip Radlinski and Nick Craswell. 2010. Comparing the sensitivity of information retrieval metrics. In *Proceedings of the 33rd international ACM SIGIR conference on Research and development in information retrieval*. 667–674.

[27] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3505–3506.

[28] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[29] Prakhar Sharma and Vinod Yegneswaran. 2023. PROSPER: Extracting Protocol Specifications Using Large Language Models. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks* (Cambridge, MA,

USA) *(HotNets '23)*. Association for Computing Machinery, New York, NY, USA, 41–47. doi:10.1145/3626111.3628205

[30] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv:2408.03314 [cs.LG] https://arxiv.org/abs/2408.03314

[31] Qwen Team. 2025. QwQ-32B: Embracing the Power of Reinforcement Learning. https://qwenlm.github.io/blog/qwq-32b/

[32] Niels ten Oever, Corinne Cath, Mirja Kühlewind, and Colin Perkins. 2022. Report from the IAB Workshop on Analyzing IETF Data (AID) 2021. RFC 9307. doi:10.17487/RFC9307

[33] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[34] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533* (2022).

[35] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2023. Improving Text Embeddings with Large Language Models. *arXiv preprint arXiv:2401.00368* (2023).

[36] Orion Weller, Benjamin Van Durme, Dawn Lawrie, Ashwin Paranjape, Yuhao Zhang, and Jack Hessel. 2024. Promptriever: Instruction-trained retrievers can be prompted like language models. *arXiv preprint arXiv:2409.11136* (2024).

[37] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597 [cs.CL]

[38] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).

[39] Jane Yen, Ramesh Govindan, and Barath Raghavan. 2021. Tools for disambiguating RFCs. In *Proceedings of the 2021 Applied Networking Research Workshop*. 85–91.

[40] Jane Yen, Tamás Lévai, Qinyuan Ye, Xiang Ren, Ramesh Govindan, and Barath Raghavan. 2021. Semi-automated protocol disambiguation and code generation. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. 272–286.

[41] Botao Yu, Frazier N. Baker, Ziqi Chen, Xia Ning, and Huan Sun. 2024. LlaSMol: Advancing Large Language Models for Chemistry with a Large-Scale, Comprehensive, High-Quality Instruction Tuning Dataset. *arXiv preprint arXiv:2402.09391* (2024).

[42] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. 2023. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792* (2023).

[43] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2019. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675* (2019).